EXATHON INDIA 2026

TRACEOPS

Trace the data. Find the cause. Measure the impact

A data observability and investigation platform: incidents, lineage, quality checks, root-cause hypotheses and KPI impact in one workflow.

Presenter name: Bryane Angelo J & Sandhiyasri M

Team name : Coders

College / institution : Easwari Engineering College

Date: 25.10.2026

TABLE OF CONTENTS

1. Problem Statement
2. Abstract
3. Existing System & Gaps
4. Requirements
5. System Architecture
6. Application Workflow
7. Data Lineage
8. Module Description
9. Features Being Developed
10. Proposed Features
11. Objectives
12. Scope
13. UI Design
14. Technology Stack
15. Implementation Challenges
16. Benefits

PROBLEM STATEMENT

Modern organizations depend on data pipelines that feed reports, dashboards, and business KPIs. When a data-quality failure occurs, it is often noticed only after a report or metric looks wrong, and by then several downstream datasets may already be affected.

Investigating such an incident requires tracing which datasets are connected, checking which quality checks failed, gathering evidence, and judging the business impact. These activities are usually carried out in separate tools and records, which makes the investigation slow and difficult to repeat.

The absence of a single system that connects incidents, lineage, quality checks, evidence, and business impact makes it difficult to find root causes quickly and to explain the importance of an incident to the business.

ABSTRACT

TraceOps is a data observability and investigation platform that helps users identify data-quality issues, trace data lineage, investigate incidents, and understand their potential root causes and business impact. The system is built with a React.js frontend and a Python FastAPI backend, with Exasol integration and demonstration data.

Users can view incidents, examine upstream and downstream lineage, review quality-check results, study investigation details with supporting evidence and root-cause hypotheses, and see the KPI impact of each incident. By bringing these activities into one workflow, TraceOps aims to make data incident investigation more organized, traceable, and easier to communicate.

EXISTING APPROACH & GAPS

CURRENT APPROACH

- Data-quality failures are often discovered through reports or KPIs that look incorrect.
- Affected datasets are identified manually by following dependencies across systems.
- Quality-check results, evidence, and business context are kept in separate places.

LIMITATIONS / GAPS

- No single view connects incidents, lineage, quality checks, and business impact.
- Tracing upstream and downstream datasets takes time and effort.
- Evidence for a probable cause is not kept together with the incident.
- The business impact of an incident is difficult to link to the data issue.
- Investigation steps are hard to repeat or explain to others.

REQUIREMENTS

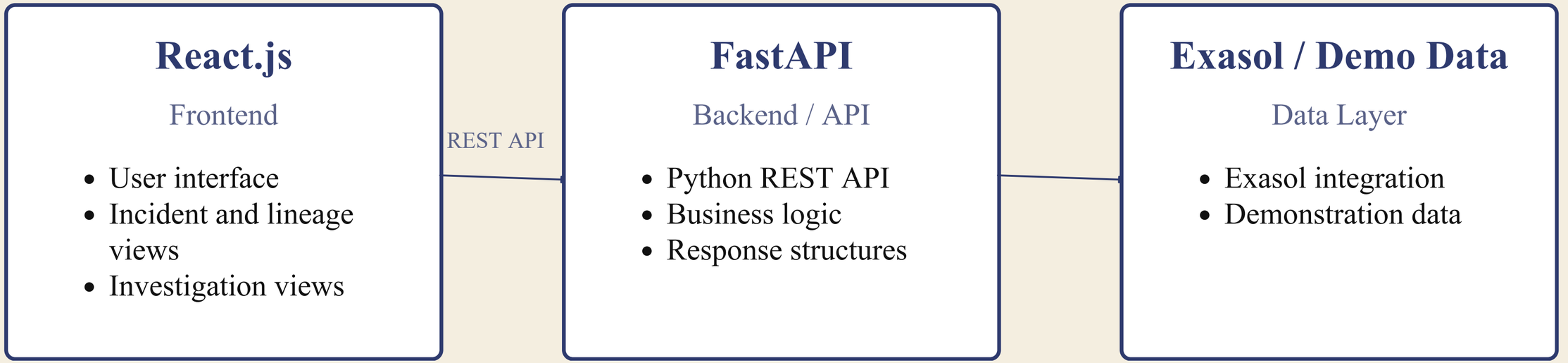
FUNCTIONAL

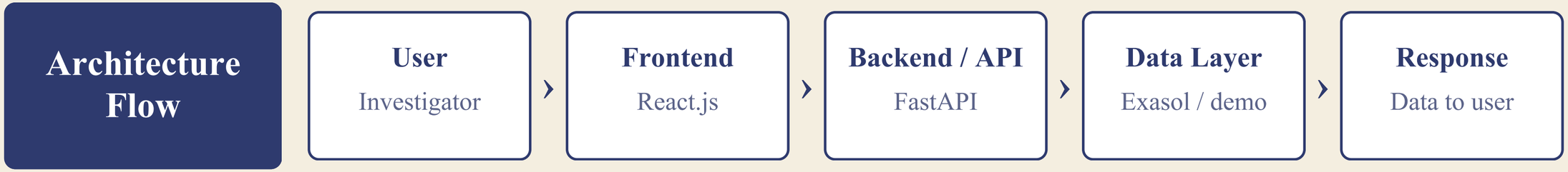
- Display incidents with their summary information.
- Show data lineage with upstream and downstream dependencies.
- Present quality-check results for each incident.
- Show investigation details, evidence, and root-cause hypotheses.
- Show the KPI impact of an incident.

NON-FUNCTIONAL

- Separation of frontend, backend, and data layers.
- REST API communication between the React frontend and the FastAPI backend.
- Consistent field names and response structure between the API and the interface.
- Clear, readable interface suitable for investigation and demonstration.

SYSTEM ARCHITECTURE





APPLICATION WORKFLOW

1

Select Incident

User opens an incident from the dashboard.

›

2

Review Lineage

Upstream and downstream datasets are examined.

›

3

Check Quality

Quality-check results for the incident are reviewed.

›

4

Investigate

Evidence and root-cause hypotheses are studied.

›

5

Assess Impact

KPI impact of the incident is reviewed.

Connect • Investigate • Understand Impact

DATA LINEAGE



- Upstream nodes show where the data originates and where a problem may begin.
- Downstream nodes show which datasets and reports may be affected.
- Arrows show the direction of data flow between nodes.

MODULE DESCRIPTION

Incident Dashboard Module

Lists the incidents and gives users the starting point for an investigation.

Data Lineage Module

Shows the nodes and edges that connect upstream and downstream datasets for an incident.

Quality Check Module

Presents the quality-check results associated with an incident.

Investigation Module

Displays investigation details, supporting evidence, and root-cause hypotheses.

KPI Impact Module

Shows the business metrics affected by an incident.

API & Data Module

FastAPI routes supply incident data from the Exasol integration or demonstration data to the frontend.

FEATURES BEING DEVELOPED

- **Incident Dashboard –** Overview of the incidents available for investigation.
- **Data Lineage –** Upstream and downstream dependencies shown as labelled nodes and edges.
- **Quality Checks –** Quality-check results and counts for each incident.
- **Investigation Details –** Incident context together with supporting evidence.
- **Root-Cause Hypotheses –** Candidate causes presented for validation.
- **KPI Impact –** Business impact of each incident.

NEW / PROPOSED FEATURES

- **Alerting –** Send notifications when new incidents are detected.
- **Incident History –** Maintain past incidents with trend views.
- **Additional Quality Checks –** Support a wider range of quality-check types.
- **Access Control –** Add authentication and role-based access.
- **Live Data Coverage –** Extend the Exasol integration beyond demonstration data.
- **Report Export –** Export incident summaries for sharing.

OBJECTIVES

The main objective of TraceOps is to provide one platform for investigating data incidents from detection to business impact. It aims to help users identify data-quality issues, trace upstream and downstream dependencies through data lineage, and review quality-check results.

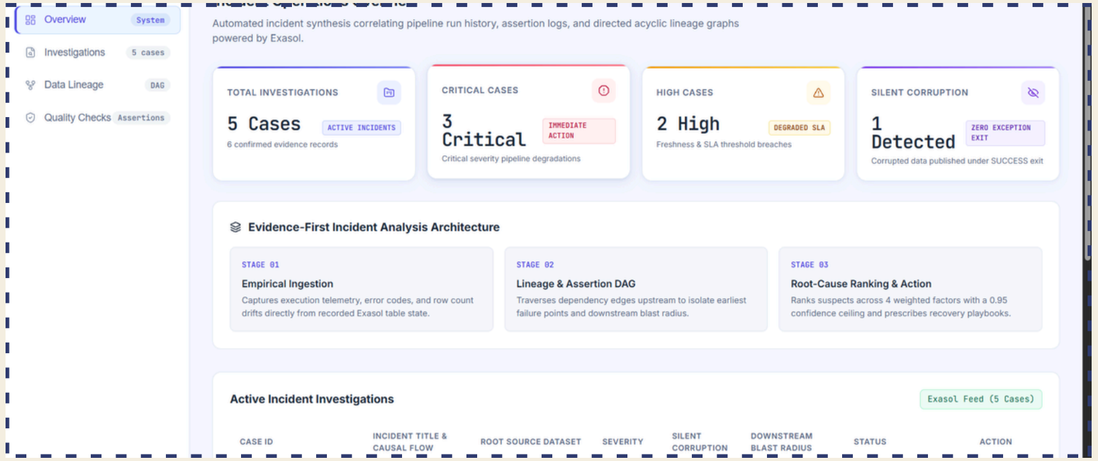
The system also presents evidence and root-cause hypotheses for each incident, and shows the KPI impact so that technical findings can be connected to business outcomes. It keeps the complete investigation in one place so that it can be reviewed and explained clearly.

SCOPE

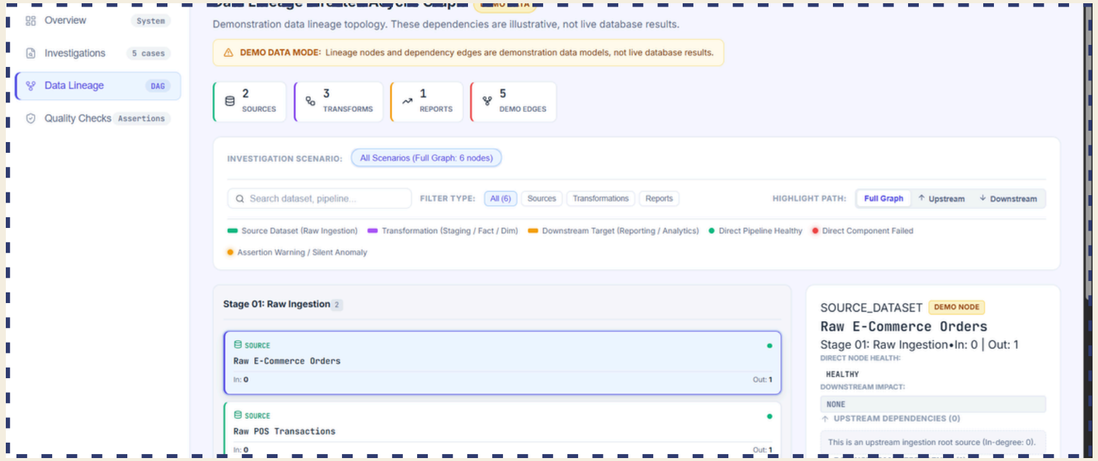
1. **Incident Management** - Incident dashboard, Incident summary
2. **Data Lineage** - Upstream and downstream datasets, Node and edge relationships
3. **Data Quality** - Quality-check results, Check counts per incident
4. **Investigation** - Investigation details, Evidence, Root-cause hypotheses
5. **Business Impact** - KPI impact, Incident summary for stakeholders
6. **Platform** - React frontend, FastAPI backend, Exasol integration and demonstration data

UI DESIGN

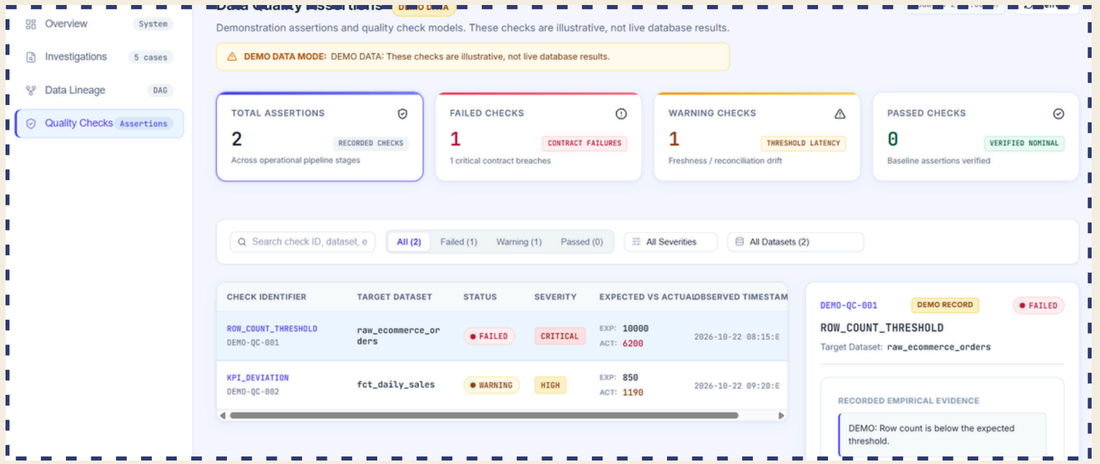
1. Incident Dashboard



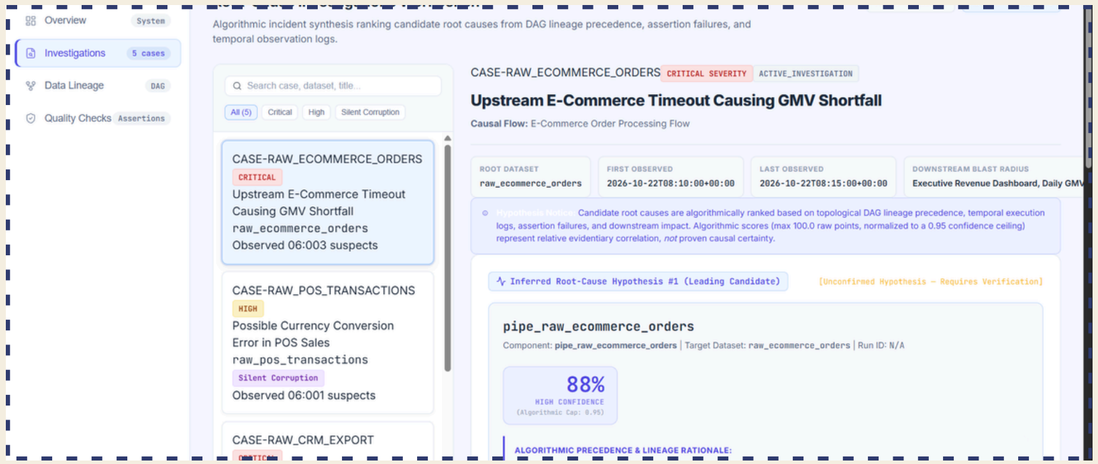
2. Data Lineage



3. Quality Checks



4. Investigation, Evidence & KPI Impact



TECHNOLOGY STACK

- **Frontend:** React.js — interactive, component-based user interface.
- **Language:** JavaScript — interface logic and API response handling.
- **Styling:** CSS — layout and appearance of the interface.
- **Backend:** Python + FastAPI — REST API and business logic.
- **Database:** Exasol — integration and demonstration data.

IMPLEMENTATION CHALLENGES

CHALLENGES

- API field names and response parsing in the frontend did not match, so lineage, counts, and investigation details displayed incorrectly.
- Synthetic incidents had been mixed in with the original two incidents.

SOLUTIONS

- Field names and parsing were aligned with the actual API response structure.
- Synthetic records were removed so that only the original incidents are shown.

BENEFITS

- Brings incidents, lineage, quality checks, evidence, and impact into one workflow.
- Shows what feeds and depends on an affected dataset, giving context for every issue.
- Links a technical data issue to the business metrics it affects.
- Keeps evidence together with each root-cause hypothesis for easier review.
- Separates frontend, backend, and data layers for easier maintenance.

THANK YOU